System Architectural Walkthrough

Restaurant Order, Table Reservation, Kitchen Queue, Deliveries & Cashier POS Monorepo

`React 19 SPA` `Laravel 12/13 API` `Supabase Auth (ES256)` `Supabase Realtime` `Chapa ETB Gateway` `PostgreSQL / SQLite` `TypeScript Monorepo (@intern/shared)` `Tailwind CSS v4`

Table of Contents

1. Executive Overview & Tech Stack
2. Monorepo & Directory Structure
3. System Entry Points & Bootstrapping
4. Core Architecture & Interaction Diagram
5. Data Layer, ERD & Eloquent Models
6. Deep-Dive Business Logic & Workflows
7. External API Surface & Route Catalog
8. Cross-Cutting Concerns & Security
9. Ecosystem Dependencies
10. Build, Run & Testing Manual
11. Architectural Observations & Gotchas

1. Executive Overview & Tech Stack Matrix

This codebase implements a full-stack, enterprise-grade restaurant management and point-of-sale system. It integrates public customer ordering (dine-in with table selection and home delivery with live dispatch), staff table reservations with automated timeslot conflict checking, expediter and kitchen queues, driver assignment, cashier payment verification (Cash and Chapa ETB online gateway), staff menu modification proposals, and administrative role-based access control (RBAC).

Layer	Technology / Framework	Architectural Purpose
Frontend SPA	React 19, TypeScript, Vite 8, Tailwind CSS v4, TanStack Query v5	Client-side rendered application with late-night bistro aesthetic, micro-animations, and responsive layouts.
Backend REST API	Laravel 12/13, PHP 8.3+, Eloquent ORM, Firebase JWT	RESTful API enforcing business rules, authorization Gates, transaction boundaries, and rate limits.
Identity & Auth	Supabase Auth (ES256 JWKS asymmetric keys)	Decoupled auth provider issuing cryptographically signed JWTs validated locally on Laravel.
Live Synchronization	Supabase Realtime (WebSocket) + 5s Polling Fallback	Live order updates for kitchen queues and dispatchers with automated degradation resilience.
Database Layer	Supabase PostgreSQL (Production) / SQLite (Dev & Tests)	Relational schema with composite indexes, foreign key constraints, and RLS security policies.
Payment Gateway	Chapa API (Ethiopian Birr / ETB) + Cash Desk	Dual payment handling: instant online checkout verification + cashier desk approval gate.
Shared Contracts	`@intern/shared` (TypeScript npm workspace)	Shared type definitions, DTO envelopes, enums, roles, and permission constants.

2. Monorepo & Directory Structure

The monorepo uses npm workspaces uniting the frontend client and shared contracts, maintaining strict domain boundary isolation from the Laravel API server.

```
intern-project/
├── client/                    # React 19 Frontend SPA (Vite + Tailwind CSS v4)
│   ├── src/
│   │   ├── components/        # Reusable UI components (ConfirmModal, Skeletons, Inputs)
│   │   ├── hooks/             # Custom hooks (useOrdersRealtime, useDebouncedValue)
│   │   ├── layouts/           # AppLayout (public/staff), AdminLayout, AuthLayout
│   │   ├── pages/             # 14 Feature Pages (OrdersPage, DeliveriesPage, PaymentsPage, etc.)
│   │   ├── services/          # HTTP API client (api.ts) & Supabase client (supabase.ts)
│   │   ├── store/             # React Context (AuthContext & RBAC permission helpers)
│   │   ├── styles/            # Global CSS, bistro theme tokens, keyframe animations
│   │   ├── utils/             # ETB currency formatters, admin routing paths
│   │   ├── App.tsx            # React Router v7 routes & ProtectedRoute guards
│   │   ├── main.tsx           # DOM root mounting & TanStack QueryClientProvider wrapper
│   │   └── queryClient.ts     # Global React Query instance with tuned stale times
├── server/                    # Laravel 12/13 REST API Application
│   ├── app/
│   │   ├── Http/Controllers/  # API Controllers (OrderController, CashierController, MenuController)
│   │   ├── Http/Middleware/   # VerifySupabaseJwt, SecurityHeaders
│   │   ├── Http/Requests/     # FormRequest classes with strict field validation rules
│   │   ├── Models/            # Eloquent Models (Order, User, MenuItem, RestaurantTable, Role)
│   │   ├── Policies/          # Granular authorization policies bound to RBAC permissions
│   │   ├── Providers/         # AppServiceProvider (Rate limiting, Gate definition, DI singletons)
│   │   ├── Services/          # SupabaseJwtService, ChapaPaymentService, ApiCacheService
│   │   └── Support/           # KeysetPaginator cursor pagination helper (zero OFFSET cost)
│   ├── database/migrations/   # 21 schema migrations with composite performance indexes
│   ├── database/seeders/      # Seeders (Admin users, Roles, Permissions, Ethiopian menu dishes)
│   ├── routes/api.php         # REST API endpoint map under throttle:api
│   └── tests/                 # PHPUnit Feature and Unit test suites
├── shared/                    # Shared TypeScript package (@intern/shared)
│   └── types/index.ts         # Strongly typed interfaces, DTOs, API envelopes, role constants
├── docs/                      # Architecture and environment setup documentation
└── package.json               # Root package workspace definition & unified scripts
```

3. System Entry Points & Lifecycle Bootstrapping

Client Bootstrap (React 19)

1. `client/index.html` mounts font stylesheets and `#root` DOM container.
2. `main.tsx` initializes React in `StrictMode` and provides the global `QueryClientProvider`.
3. `App.tsx` encapsulates navigation with `BrowserRouter` and mounts `AuthProvider`.
4. `AuthContext.tsx` triggers `supabase.auth.getSession()`, listens to `onAuthStateChange`, and queries `GET /api/me` to populate user permissions.
5. Protected routes are gated via `ProtectedRoute` checking session and permission tokens.

Server Bootstrap (Laravel 12/13)

1. `server/public/index.php` receives HTTP requests and invokes the Laravel kernel.
2. `bootstrap/app.php` registers routing, configures global `SecurityHeaders` middleware, and sets JSON exception formatting for `api/*`.
3. `AppServiceProvider.php` registers singletons (`ApiCacheService`, `SupabaseJwtService`), defines rate limits, and dynamically registers Gates from `Permission::catalog()`.
4. `routes/api.php` processes endpoints through `throttle:api` and `VerifySupabaseJwt`.

4. Core Architecture & Component Interaction

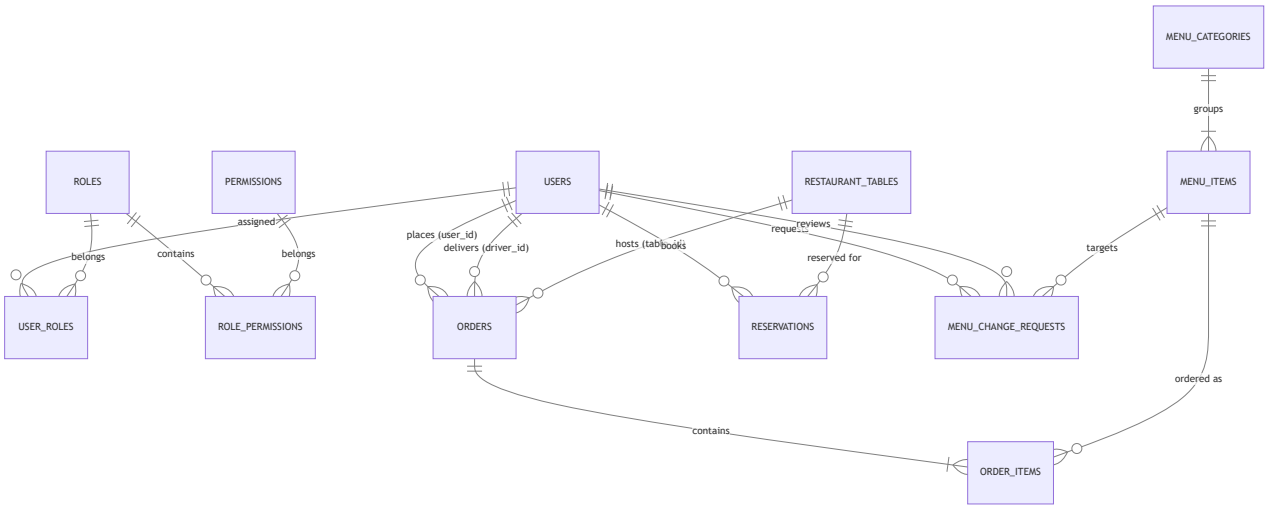
The system decouples identity management and real-time pub/sub onto Supabase infrastructure while maintaining complete business logic, transaction boundaries, cashier settlement gates, and data persistence inside Laravel and PostgreSQL.

FIGURE 1: HIGH-LEVEL SYSTEM ARCHITECTURE & DATA FLOW



5. Data Layer, ERD & Eloquent Models

FIGURE 2: ENTITY RELATIONSHIP DIAGRAM



Key Database Entities

- `User` : Uses UUID primary keys mapped directly to Supabase Auth `sub` . Maintains a many-to-many relationship with `Role` . Auth profiles and effective permissions are cached in-memory and in cache store via `ApiCacheService` .
- `Order` : Holds fulfillment type (`dine_in` , `delivery`), table reference, delivery coordinates/phone/notes, assigned driver, payment method (`cash` , `chapa`), payment status, and dual cashier validation timestamps.
- `OrderItem` : Normalized line items capturing immutable snapshots of unit prices and computed line totals.
- `RestaurantTable` : Physical restaurant table records containing seat capacities and operational statuses (`free` , `reserved` , `occupied`).
- `Reservation` : Table bookings capturing `party_size` , start/end timestamps, and reservation status (`confirmed` , `cancelled`).
- `MenuCategory` & `MenuItem` : Menu taxonomy with sort ordering, prices in ETB, availability flags, and image storage paths.

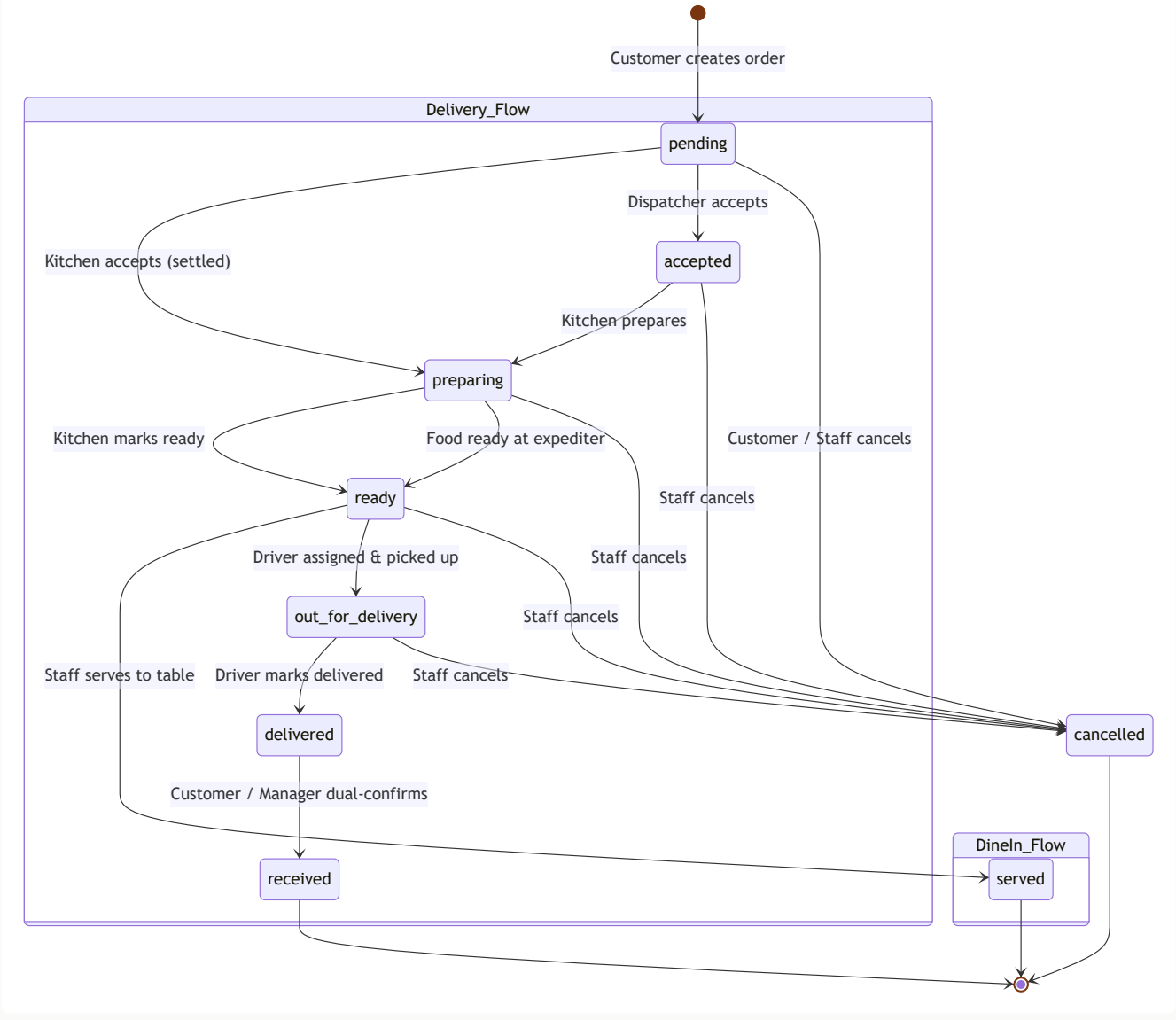
- `MenuChangeRequest`: Moderation queue for staff proposals (creation or deletion of menu items) awaiting manager approval.
- `StaffNotification`: In-app operational notification queue alerting cashiers and managers to pending actions.

6. Deep-Dive Business Logic & Workflows

Workflow 1: Dual-Fulfillment Order State Machine

Orders follow strict state progression modeled in `Order.php` and enforced in `OrderController::updateStatus`. Dine-in and Delivery paths branch dynamically with distinct permission gates.

FIGURE 3: ORDER STATE MACHINE TRANSITIONS



Critical Dispatch Invariant

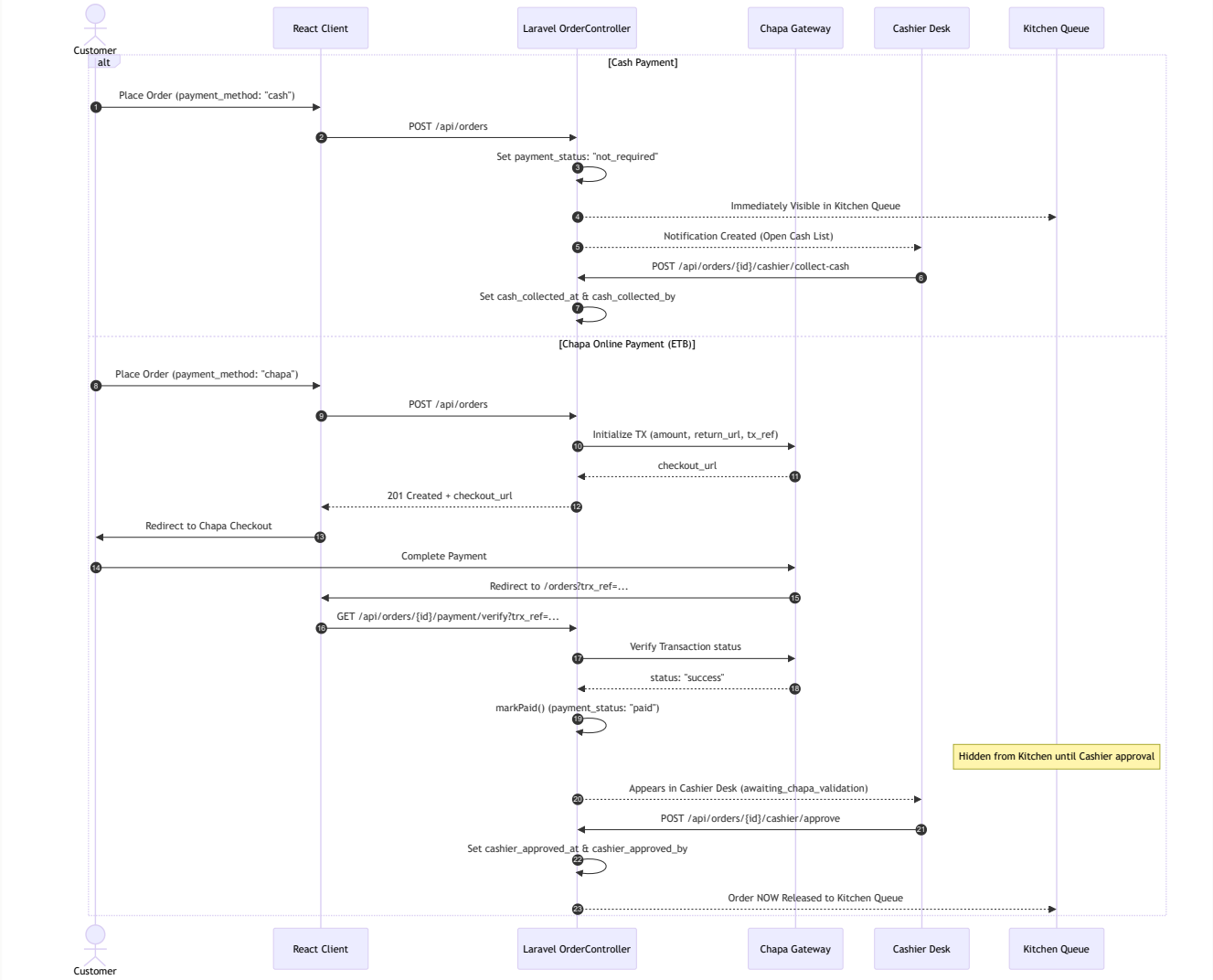
An order cannot transition to `out_for_delivery` unless a valid `driver_id` is assigned. Only the assigned driver or users with `can_manage_deliveries` can mark an order delivered.

Workflow 2: Hybrid Payment Settlement & Cashier Approval Gate

To prevent unpaid orders from consuming kitchen resources while supporting immediate cash seating, the kitchen queue enforces the `scopePaymentSettled` database scope:

```php
// App\Models\Order::scopePaymentSettled
public function scopePaymentSettled($query)
{
    return $query->where(function ($builder) {
        $builder
            ->where('payment_status', self::PAYMENT_NOT_REQUIRED) // Cash orders
            ->orWhere(function ($chapa) {
                $chapa
                    ->where('payment_status', self::PAYMENT_PAID)    // Online Chapa paid
                    ->whereNotNull('cashier_approved_at');            // AND approved by cashier!
            });
    });
}
```

FIGURE 4: PAYMENT VERIFICATION & CASHIER APPROVAL SEQUENCE



Workflow 3: Supabase JWT Verification & JIT User Synchronization

1. **Authorization Header Extraction**: Extracts bearer token from `Authorization: Bearer <token>`.
2. **Algorithm Header Inspection**: Determines algorithm (`ES256` vs legacy/test `HS256`).
3. **JWKS Verification**: For `ES256`, fetches and caches Supabase JWKS from `/.well-known/jwks.json` for 24 hours. Validates cryptographic signature using `firebase/php-jwt`. Falls back to `/auth/v1/user` API on connection issues.
4. **Claims Cache**: Caches valid JWT claims for 90 seconds (`auth.jwt.<hash>`) to avoid redundant cryptographic decoding.

5. **JIT Local Synchronization**: Upserts the user record into the local `users` table matching `id = claims.sub`. New sign-ups are automatically granted the `customer` role.
6. **User Context Injection**: Attaches resolved `User` instance to `Auth::setUser($user)` and `$request->setUserResolver()`.

Workflow 4: Table Reservation & Conflict Prevention Engine

Managed by `ReservationController` and `ReservationAvailabilityService` using row-level locking:

1. Executes inside a database transaction with `RestaurantTable::lockForUpdate()`.
2. Validates that `party_size ⩽ table.capacity`.
3. Asserts that no conflicting reservation exists within the requested window: `starts_at < $endsAt AND ends_at > $startsAt AND status = 'confirmed'`.
4. Creates the confirmed reservation and flushes the admin dashboard analytics cache.

7. External API Surface & Route Catalog

All responses return the standardized `ApiResponse<T>` envelope (`{ success, data, message?, meta? }`).

Method	Endpoint	Authorization / Policy	Description
`GET`	`/api/health`	Public	Service heartbeat probe.
`GET`	`/api/menu`	Public (ETag Cached)	Active menu categories and available items.
`GET`	`/api/tables`	Public (ETag Cached)	Physical table list and status.
`GET`	`/api/payments/chapa/callback`	Throttle: `payments`	Browser return callback from Chapa.
`POST`	`/api/payments/chapa/webhook`	Throttle: `payments`	Asynchronous IPN webhook from Chapa.
`GET`	`/api/me`	Authenticated User	User profile, assigned roles, and permissions.
`GET`	`/api/orders`	Authenticated User	Cursor-paginated personal orders (Keyset).
`POST`	`/api/orders`	`can_place_order`	Place Dine-In or Delivery order.
`GET`	`/api/orders/queue`	`can_view_all_orders`	Kitchen prep queue (settled orders only).
`GET`	`/api/orders/{id}/receipt`	Owner / `can_view_all_orders`	Order receipt payload.
`GET`	`/api/orders/{id}/payment/verify`	Order Owner	Verify Chapa transaction status.
`PATCH`	`/api/orders/{id}/status`	Dynamic FSM Gate	Transition order status along lifecycle.
`PATCH`	`/api/orders/{id}/driver`	`can_manage_deliveries`	Assign delivery courier to order.
`DELETE`	`/api/orders/{id}`	Owner (if pending) / Admin	Cancel active order.
`GET`	`/api/deliveries`	`can_manage_deliveries`	Active deliveries dispatch list.
`GET`	`/api/deliveries/drivers`	`can_manage_deliveries`	Available couriers list.
`GET`	`/api/payments/desk`	`can_manage_payments`	Cashier desk (awaiting Chapa / open cash).
`POST`	`/api/orders/{id}/cashier/approve`	`can_manage_payments`	Cashier approves Chapa payment.
`POST`	`/api/orders/{id}/cashier/collect-cash`	`can_manage_payments`	Mark cash collected for order.
`GET`	`/api/menu/manage`	`can_manage_menu` / `can_propose`	Full menu catalog (including unavailable items).
`POST`	`/api/menu-change-requests`	`can_propose_menu_changes`	Submit proposal to create/delete menu item.
`PATCH`	`/api/menu-change-requests/{id}/approve`	`can_manage_menu`	Approve and apply menu change.
`POST`	`/api/reservations`	`can_create_reservation`	Book table reservation with conflict lock.
`GET`	`/api/admin/dashboard`	`can_manage_permissions`	Revenue, sales, and operations KPI metrics.

Method	Endpoint	Authorization / Policy	Description
`GET`	`/api/admin/roles`	`can_manage_permissions`	Role catalog with assigned permissions.
`PUT`	`/api/admin/users/{user}/roles`	`can_manage_permissions`	Assign roles to user account.

8. Cross-Cutting Concerns & Security Architecture

Two-Tier Caching Architecture (`ApiCacheService.php`)

- **Tier 1: Request-Lifecycle Static RAM Cache (0ms)**: Process-level memory array `private static array $memoryCache` caching entities and permission resolutions within the same request lifecycle.
- **Tier 2: Persistent Cache Store**: Caches menu payloads (`api.menu.public.v2`), tables catalog, roles catalog, driver lists, and computed admin dashboards with sha256 date-range keys.
- **HTTP ETag Negotiation**: `/api/menu` and `/api/tables` compute md5 payload hashes and return `304 Not Modified` when matched against `If-None-Match` headers.

Keyset (Cursor) Pagination (`KeysetPaginator.php`)

Replaces standard SQL `OFFSET / LIMIT` with `WHERE id > :after_id LIMIT n + 1`. This prevents full-table scan degradation on high-volume order and reservation tables.

Rate Limiting & Security Headers

- **Rate Limiters**: `api` (120/min per user/IP), `orders` (30/min), `uploads` (15/min), `payments` (60/min per IP).
- **Security Headers**: Injected on all responses: `X-Content-Type-Options: nosniff` , `X-Frame-Options: SAMEORIGIN` , `X-XSS-Protection: 1; mode=block` , `Referrer-Policy: strict-origin-when-cross-origin` .

9. Key Ecosystem Dependencies

Frontend Dependencies

- `react` & `react-dom` **(19.2.x)**: Core UI rendering with React 19 concurrent features.
- `react-router-dom` **(7.18.x)**: SPA client routing and nested layout trees.
- `@tanstack/react-query` **(5.101.x)**: Server state cache, debounced prefetching, and query invalidation.
- `@supabase/supabase-js` **(2.110.x)**: Client auth session and Realtime WebSocket subscriptions.
- `@tailwindcss/vite` & `tailwindcss` **(4.3.x)**: Zero-config CSS utility engine with custom bistro color tokens.

Backend Dependencies

- `laravel/framework` **(13.x/12.x)**: Core backend framework providing routing, Eloquent ORM, and Gates.
- `firebase/php-jwt` **(7.1.x)**: Asymmetric JWKS and ES256 Elliptic Curve JWT signature verification.
- `phpunit/phpunit` **(12.5.x)**: Automated test execution suite for feature and unit tests.
- `mockery/mockery` **(1.6.x)**: Mocking framework for HTTP and external gateway isolation.

10. Local Installation, Build & Testing Manual

Prerequisites

- Node.js 20+ and npm 10+
- PHP 8.3+ with extensions: `openssl`, `curl`, `mbstring`, `pdo_sqlite`, `pdo_pgsql`, `fileinfo`
- Composer 2+

Step-by-Step Execution

```
# 1. Install dependencies across all monorepo packages
npm install
cd server && composer install && cd ..

# 2. Configure environment variables
cp .env.example .env
cp client/.env.example client/.env
cp server/.env.example server/.env

# 3. Migrate and seed the database
cd server
php artisan migrate --seed
cd ..

# 4. Start the development servers in two terminals:
# Terminal 1: Laravel API Backend (:8000)
cd server && php artisan serve

# Terminal 2: React SPA Frontend (:5173)
npm run dev

# 5. Run tests & build validation
cd server && php artisan test       # Backend test suite
npm run build                       # Build TypeScript shared contracts and client SPA
npm run lint                        # Lint shared and client packages
```

11. Architectural Observations, Risks & Mitigations

Area	Architectural Observation / Gotcha	Implemented Mitigation
JWKS Network Overhead	Validating ES256 tokens against remote Supabase JWKS could introduce network latency if fetched per request.	`SupabaseJwtService` caches JWKS for 24 hours (`86400s`) and caches decoded JWT claims in Redis/File cache for 90 seconds.
Windows PHP SSL Certificates	PHP on Windows environments may fail HTTPS requests to Supabase or Chapa if `curl.cainfo` is omitted.	`SupabaseJwtService::resolveCaBundle` automatically checks `services.supabase.ssl_cert_file` , `curl.cainfo` , and `openssl.cafile` paths.
Chapa Webhook in Local Dev	Chapa requires a public HTTPS URL for asynchronous webhook notifications.	`ChapaPaymentService` detects localhost origins, avoids dispatching local webhook URLs, and relies on client return verification (`/orders/{id}/payment/verify`).
PostgreSQL vs SQLite DDL	PostgreSQL-specific RLS policies (`ALTER TABLE ... ENABLE ROW LEVEL SECURITY`) fail on SQLite in local tests.	Migration `2026_08_14_180100_add_supabase_rls_policies.php` checks `DB::getDriverName() === 'pgsql'` prior to executing PostgreSQL DDL.
Realtime Channel Fallback	Local SQLite does not publish PostgreSQL replication change feeds to Supabase Realtime.	`useOrdersRealtime.ts` detects unestablished Realtime connections within 2.5s and automatically switches to 5-second polling.